

DOCUMENTACIÓN TÉCNICA INTEGRAL

Presentia

Módulo de registro de jornada para Expira

Versión 1.0.0

Fecha 2026-07-19

Ámbito Revisión Fase 1



Índice

Resumen ejecutivo

Arquitectura

Modelo de datos

API

Lógica de negocio

Manager, pantalla por pantalla

Kiosko

Roles y permisos

Configuración

Diseño y temas

Flujos de uso

Integración en Expira

Glosario

Observaciones / deuda

1. Resumen ejecutivo

Presentia es un módulo de **registro de jornada laboral** (control horario, art. 34.9 ET) pensado para integrarse dentro de un software de gestión llamado **Expira**, del que hoy **no existe código en este repositorio** (fue eliminado; archivado en el tag git `presentia-original-preborrado`). Por ello el módulo se ha construido contra una **interfaz de puertos explícita** (`src/ports.js`): Expira deberá inyectar `db`, `clock`, `employees`, `pin`, `session` y, opcionalmente, `correlatives`, `printing` y `hash`. El módulo no tiene login propio, no crea procesos ni bases de datos nuevas: añade 9 tablas con prefijo `presentia_` a la base SQLite que ya posea Expira, mediante una migración **aditiva e idempotente** (`CREATE TABLE IF NOT EXISTS`).

El backend es JavaScript puro sobre Node ≥ 22 (probado en Node 24), usando únicamente builtins (`node:sqlite`, `node:crypto`) — **cero dependencias de producción** (`package.json` no declara ninguna). La capa HTTP es agnóstica de framework: los *handlers* (`src/http/handlers.js`) son funciones puras `(deps, ctx) => resultado` que un adaptador delgado (`src/http/fastify-adapter.js`) cuelga de la instancia Fastify que aporte Expira bajo el prefijo `/presentia`.

Funcionalmente cubre: fichaje de entrada/salida por PIN desde un **kiosko** táctil (con pausas, turnos que cruzan medianoche y código correlativo `F-AAAA-NNNN`), un **Manager** con 6 pestañas (Hoy, Registros, Informe de horas, Solicitudes, Ajustes, Legal) para el rol `local_admin` / `technician`, exportación **CSV y PDF** sin dependencias, un modelo de corrección de errores **sin edición destructiva** (las correcciones generan versiones que conservan el valor original) y una **auditoría append-only encadenada por hash** que detecta alteraciones. Incluye 10 documentos legales (RGPD, LSSI, EULA) embebidos y un sistema de temas claro/oscuro/automático con tokens CSS centralizados.

El estado de la suite de pruebas se ha verificado en esta revisión ejecutando `npm test`: **121/121 pruebas en verde** (dominio, migración, flujos de negocio, seguridad, legal, tema, responsive, release). El servidor de previsualización (`dev-preview/api-server.mjs`) está operativo en `http://127.0.0.1:8787` sobre el entorno de referencia en memoria, con 4 empleados de demostración (Ana/`e1`, Bruno/`e2`, Laura/`a1` admin, Tec/`t1` técnico).

El módulo se declara "apto para integrar" por su propio equipo (`ENTREGA.md`), pero con condiciones reales pendientes: cifrado en reposo, TLS, empaquetado de la tipografía Inter (nunca cargada hoy), relleno de los marcadores legales (`[NOMBRE_EMPRESA]`, etc.) y, sobre todo, validar el módulo contra el código **real** de Expira, que no ha podido probarse porque no existe en este entorno. Esta revisión añade hallazgos propios más allá de los que el propio equipo ya documentó (ver §14), entre ellos que las claves foráneas del esquema no se hacen cumplir (no hay `PRAGMA foreign_keys = ON` en ningún punto) y que la cadena de auditoría no detecta el truncamiento de sus últimas filas, sólo la alteración de filas intermedias.

2. Arquitectura

2.1 Principio rector

Presentia **no es una aplicación independiente**: es código fuente que Expira importa y monta. No abre su propio puerto de red, no tiene su propio proceso, no gestiona su propia base de datos y no implementa login. Toda dependencia externa se recibe **inyectada** como "puerto" (patrón *ports & adapters* / arquitectura hexagonal).

2.2 Puertos consumidos de Expira (`src/ports.js`)

Puerto	Obligatorio	Interfaz	Si falta
<code>db</code>	Sí	<code>{ exec(sql), prepare(sql) → {run,get,all} }</code> , compatible <code>node:sqlite</code>	<code>crearModulo</code> lanza error explícito
<code>clock.now</code>	Sí	<code>() → epoch_ms</code>	lanza error
<code>employees</code>	Sí	<code>getById(id) → Empleado null</code> , <code>list()</code> → <code>Empleado[]</code>	lanza error

Puerto	Obligatorio	Interfaz	Si falta
<code>session.resolve</code>	Sí	<code>(req) → {empleadoId, rol} null</code>	lanza error
<code>pin.verify</code>	Sí si <code>exigirPin</code> (por defecto sí)	<code>(empleadoId, pin) → boolean</code> , tiempo constante	lanza error si <code>exigirPin</code> activo
<code>correlativos.next</code>	No	<code>(serie, anio) → 'F-AAAA-NNNN'</code>	fallback: contador atómico propio en <code>presentia_correlativos</code>
<code>printing</code>	No	<code>printTicket(doc)</code> , <code>renderPdf(report) → Buffer</code>	ticket no se imprime; PDF usa generador propio
<code>hash</code>	No	<code>hashSecret(s)</code> , <code>verifySecret(s, stored)</code>	fallback <code>scrypt</code> propio (<code>src/security/hash.js</code>)

`assertDeps()` valida en caliente que los puertos obligatorios existen y son funciones; si no, `crearModulo()` lanza con un mensaje que nombra el puerto ausente. `Empleado` esperado: `{id, nombre, rol, activo, avatarUrl?}` con `rol ∈ {empleado, local_admin, technician}` (no hay "supervisor" ni "superadmin"; ver `ROLES` en `ports.js`).

2.3 Capas del código

```

src/
  ports.js          contrato de integración, ROLES, DEFAULT_CONFIG, normalizeConfig, as
  index.js          crearModulo(deps) → { deps, rate, kioskSessions } + registrarFastify
  db/
    schema.js       DDL (9 tablas presentia_*, índices)
    migrate.js       migrate() aditivo + leerConfig/guardarConfig
    tx.js            transaccion(db, fn) vía SAVEPOINT anidable
  domain/           lógica PURA, sin BD ni red (testeable sin mocks)
    time.js          zona horaria del centro vs. timestamps UTC
    jornadas.js      emparejado de marcas, minutos, redondeo, resumen
    correlativo.js   formato/parseo F-AAAA-NNNN + contador atómico
  services/         orquestan dominio + repos + auditoría (con efectos: BD, reloj)
    repos.js, fichaje.service.js, hoy.service.js, registros.service.js,
    informe.service.js, solicitudes.service.js, ajustes.service.js,
    audit.service.js, aceptacion.service.js
  http/
    handlers.js      handlers puros (deps, ctx) → datos | lanza ErrorPresentia
    authz.js         requireRol, requireCanalKiosko, rate limiter en memoria
    kiosk-session.js micro-sesión de kiosko (token efímero, hash almacenado)
    fastify-adapter.js registrarFastify(): cuelga los handlers de Fastify bajo /presentia
  security/
    hash.js          scrypt (fallback), verificación en tiempo constante
    pin-policy.js     PIN débil/trivial, backoff exponencial
  export/
    csv.js, pdf.js   exportación sin dependencias (BOM UTF-8, PDF de texto plano)
    errors.js        ErrorPresentia (code, status, mensaje interno, mensaje público)
    dev/reference-env.js entorno de referencia SOLO dev/test (bloqueado si NODE_ENV=product
  manager/          React 19 – Manager (6 pestañas)
  kiosk/            React 19 – kiosko (fichar, mis registros, legal)
  shared/           tokens.css, temas, Markdown seguro, responsive.css
  legal/            10 documentos + generador de contenido embebido
  test/             121 pruebas (node:test, cero dependencias)
  dev-preview/      arnés de desarrollo (Vite + servidor HTTP mínimo), NO es producción
  auditoria/        catálogo de auditoría funcional previa (defectos, evidencias)

```

2.4 Flujo de arranque (`crearModulo`)

1. Valida que `deps.db` y `deps.clock.now` existan (chequeo temprano, antes de tocar BD).
2. `migrate(deps.db)` : ejecuta el DDL completo (`CREATE TABLE/INDEX IF NOT EXISTS`) y siembra `presentia_ajustes`

con `INSERT OR IGNORE` (nunca sobrescribe valores ya guardados). Ejecutarlo dos veces es no-op (probado).

1. Fusiona la config persistida (`leerConfig`) con la que pase Expira en `deps.config` , y normaliza (`normalizeConfig`).
2. Instancia `correlatives` (si Expira no lo aporta, usa el contador atómico propio).
3. `assertDeps(full)` — última comprobación con la config ya resuelta (p. ej. exige `pin.verify` si `exigirPin` es true).

4. Crea el *rate limiter* (ventana de 60 s, máx. 30 peticiones por clave) y el emisor de micro-sesiones de kiosko

(TTL 90 s).

1. Devuelve `{ deps, rate, kioskSessions }`, que se pasa a `registrarFastify(fastify, modulo, { prefix })`.

2.5 Por qué el diseño es agnóstico de Fastify (D-006)

Los handlers de `handlers.js` no reciben `req` / `reply`: reciben un objeto `ctx` plano (`{actor, canal, params, query, body, dispositivo, rate, kioskSessions, formato}`) y devuelven datos o lanzan `ErrorPresentia`. Esto permite testear las 121 pruebas sin arrancar un servidor HTTP real ni depender de Fastify como dependencia de test, y hace portable la lógica de autorización/validación a cualquier framework futuro.

3. Modelo de datos

Las 9 tablas viven en la **misma base SQLite** que Expira (puerto `db`); todas con prefijo `presentia_` para evitar colisión de nombres. El módulo nunca crea su propio fichero `.db`. En el entorno de referencia de desarrollo/test la base es `:memory:` (no existe `.db` real en disco en este repositorio).

3.1 `presentia_ajustes` — configuración clave/valor

Campo	Tipo	Notas
<code>clave</code>	TEXT	PK. Nombre del ajuste (p. ej. <code>zonaHoraria</code>)
<code>valor</code>	TEXT NOT NULL	Valor serializado en JSON (<code>JSON.stringify</code>)

Una fila por cada clave de `DEFAULT_CONFIG` (10 claves). Se lee entera y se normaliza con `normalizeConfig()` en cada `leerConfig()`.

3.2 `presentia_jornadas` — un día de un empleado

Campo	Tipo	Notas
<code>id</code>	INTEGER	PK autoincremental
<code>empleado_id</code>	TEXT NOT NULL	id del empleado (externo; no hay tabla de empleados en este módulo)
<code>fecha</code>	TEXT NOT NULL	<code>YYYY-MM-DD</code> en la zona horaria del centro , no UTC (día de la marca de entrada)
<code>codigo</code>	TEXT NOT NULL UNIQUE	<code>F-AAAA-NNNN</code> (serie configurable, año, correlativo de 4+ dígitos)

Campo	Tipo	Notas
<code>estado</code>	TEXT NOT NULL DEFAULT <code>abierta</code>	<code>CHECK IN ('abierta', 'cerrada')</code>
<code>editado</code>	INTEGER NOT NULL DEFAULT 0	1 si alguna marca de la jornada fue editada/añadida por un admin
<code>creado_ts</code> / <code>actualizado_ts</code>	INTEGER NOT NULL	epoch ms UTC
—	—	<code>UNIQUE (empleado_id, fecha)</code> : un empleado sólo puede tener una jornada por fecha de calendario del centro

3.3 `presentia_marcas` — cada fichaje individual

Campo	Tipo	Notas
<code>id</code>	INTEGER	PK autoincremental
<code>jornada_id</code>	INTEGER NOT NULL	FK declarada → <code>presentia_jornadas(id)</code> (ver nota de integridad en §14)
<code>tipo</code>	TEXT NOT NULL	<code>CHECK IN ('entrada', 'salida')</code>
<code>ts</code>	INTEGER NOT NULL	epoch ms UTC (nunca hora local)
<code>origen</code>	TEXT	<code>'kiosk' 'manager'</code> (libre, sin <code>CHECK</code>)
<code>dispositivo</code>	TEXT	identificador libre del dispositivo/kiosko
<code>editado</code>	INTEGER NOT NULL DEFAULT 0	1 si esta marca concreta fue creada/editada por un admin (no por fichaje normal)
<code>creado_ts</code>	INTEGER NOT NULL	epoch ms

Índice: `idx_presentia_marcas_jornada (jornada_id)`.

3.4 `presentia_marca_versiones` — historial inalterable (append-only)

Campo	Tipo	Notas
<code>id</code>	INTEGER	PK
<code>marca_id</code>	INTEGER NOT NULL	referencia lógica a <code>presentia_marcas.id</code> (sin <code>FOREIGN KEY</code> declarada)
<code>campo</code>	TEXT NOT NULL	hoy siempre <code>'ts'</code> (única propiedad versionable)
<code>valor_anterior</code> / <code>valor_nuevo</code>	TEXT	timestamps serializados a texto
<code>motivo</code>	TEXT NOT NULL	obligatorio en toda edición
<code>autor_id</code>	TEXT NOT NULL	quién corrigió

Campo	Tipo	Notas
ts	INTEGER NOT NULL	cuándo se corrigió

Nunca se hace `UPDATE` / `DELETE` sobre esta tabla: sólo `INSERT`. Cada corrección de una marca inserta aquí el valor **anterior**, y sólo entonces actualiza `presentia_marcas.ts` con el nuevo valor. Índice: `idx_presentia_versiones_marca`.

3.5 `presentia_solicitudes` — ciclo de vida de una corrección propuesta por el empleado

Campo	Tipo	Notas
id	INTEGER	PK
empleado_id	TEXT NOT NULL	quién la propone
tipo	TEXT NOT NULL DEFAULT 'correccion'	único valor usado hoy
jornada_id / marca_id	INTEGER	referencias lógicas opcionales (sin <code>FK</code>)
cambio	TEXT NOT NULL	JSON <code>{accion:'editar' 'anadir', marcaId?/jornadaId?, tipo?, ts}</code>
motivo	TEXT NOT NULL	obligatorio
estado	TEXT NOT NULL DEFAULT 'pendiente'	<code>CHECK IN ('pendiente', 'aprobada', 'rechazada')</code>
resuelto_por / resuelto_ts / comentario	—	rellenos al aprobar/rechazar
creado_ts	INTEGER NOT NULL	—

Índice: `idx_presentia_solici_estado`.

3.6 `presentia_auditoria` — log append-only encadenado por hash

Campo	Tipo	Notas
id	INTEGER	PK
ts, actor_id, actor_rol, accion, entidad, entidad_id, detalle (JSON), origen, motivo	—	payload del evento; <code>detalle</code> nunca contiene PIN/hash/token (verificado por test)
prev_hash	TEXT NOT NULL	hash de la fila anterior (<code>'GENESIS'</code> si es la primera)
hash	TEXT NOT NULL	<code>sha256(prev_hash + JSON-canónico(payload))</code>

No hay ningún `UPDATE` / `DELETE` sobre esta tabla en todo el código de producción. `verificarIntegridad(db)` recorre la tabla en orden de `id` y recalcula la cadena; si una fila fue alterada, `prev_hash` / `hash` dejan de cuadrar y devuelve `{ok:false, rotoEn:<id>}`. Ver limitación real en §14 (no detecta el borrado de las últimas filas).

3.7 `presentia_correlativos` — contador atómico F-AAAA-NNNN

Campo	Tipo	Notas
<code>serie</code> , <code>anio</code>	TEXT / INTEGER	PK compuesta
<code>ultimo</code>	INTEGER NOT NULL DEFAULT 0	último número asignado

Se actualiza con un único `UPSERT ... RETURNING` (`INSERT ... ON CONFLICT DO UPDATE SET ultimo = ultimo + 1 RETURNING ultimo`), atómico en una sola sentencia SQL. Fallback usado sólo si Expira no aporta su propio puerto `correlativos`.

3.8 `presentia_aceptaciones` — consentimiento de términos, una vez por versión

Campo	Tipo	Notas
<code>empleado_id</code> , <code>version</code>	TEXT	PK compuesta
<code>ts</code>	INTEGER NOT NULL	cuándo aceptó
<code>origen</code>	TEXT	'kiosk' 'manager'

`TERMINOS_VERSION = 'v1'` está fijado en código (`aceptacion.service.js`); subir esta constante fuerza a todos los usuarios a re-aceptar.

3.9 `presentia_pin_intentos` — bloqueo/backoff por empleado y dispositivo

Campo	Tipo	Notas
<code>empleado_id</code> , <code>dispositivo</code>	TEXT	PK compuesta
<code>fallos</code>	INTEGER NOT NULL DEFAULT 0	contador de intentos fallidos consecutivos
<code>bloqueado_hasta</code>	INTEGER NOT NULL DEFAULT 0	epoch ms hasta el que está bloqueado
<code>actualizado_ts</code>	INTEGER NOT NULL	—

Se limpia (`DELETE`) tras un PIN correcto; es la única tabla del módulo donde `DELETE` es una operación normal (no afecta a datos de jornada ni auditoría).

3.10 Diagrama de relaciones (ASCII)

Expira (externo, puerto `employees`)



4. API

Base por defecto: `/presentia` (configurable en `registrarFastify(fastify, modulo, { prefix })`). Sobre de respuesta uniforme: `{ ok:true, data }` en éxito, `{ ok:false, error:{ code, mensaje } }` en error (mensaje siempre genérico/público, nunca el mensaje interno). Las descargas (CSV/PDF) devuelven el binario con `Content-Disposition: attachment`.

4.1 Canal *kiosk* (identidad = PIN + micro-sesión; no depende del rol de la sesión del host)

Método · Ruta	Body/Query	Validaciones	Respuesta / código	Rol host requerido
GET /kiosk/empleados	—	ninguna (sólo canal <code>kiosk</code>)	<code>[{id,nombre,avatarUrl}]</code> ; <code>[]</code> si <code>mostrarEnKiosko=false</code>	ninguno (canal <code>kiosk</code>)
POST /kiosk/entrar	<code>{empleadoId, pin}</code>	<code>empleadoId</code> requerido (400 <code>EMPLEADO_REQUERIDO</code>); rate-limit por <code>dispositivo+empleadoId</code> (429 <code>RATE</code>); PIN vía <code>pin.verify</code> (401 <code>PIN_INCORRECTO</code> , 429 <code>PIN_BLOQUEADO</code>)	<code>{token, empleado, estado, aceptado}</code>	ninguno
POST /kiosk/estado	<code>{token}</code>	token de micro-sesión válido (401 <code>SESSION_KIOSKO</code>)	<code>{empleado, estado}</code>	ninguno
POST /kiosk/fichar	<code>{token}</code>	token válido; rate-limit por <code>dispositivo+empleadoId</code> ; <code>variasMarcasDia=false</code> + jornada ya cerrada ⇒ 409 <code>JORNADA_CERRADA</code>	<code>{tipo, ts, codigo, estado, marcaId, jornadaId}</code>	ninguno
POST /kiosk/mis-registros	<code>{token, desde?, hasta?}</code>	token válido; rango por defecto = mes en curso	informe sólo del propio empleado (mismo shape que <code>manager.informe</code>)	ninguno
GET /kiosk/mis-registros-horas.csv	? <code>token&desde&hasta</code>	token válido	CSV (BOM UTF-8, <code>;</code>)	ninguno
GET /kiosk/mis-registros-horas.pdf	? <code>token&desde&hasta</code>	token válido	PDF (texto plano, generador propio)	ninguno
POST /kiosk/solicitud	<code>{token, cambio, motivo, jornadaId?, marcaId?}</code>	token válido; <code>motivo</code> no vacío (400 <code>MOTIVO_REQUERIDO</code>); <code>cambio</code> objeto válido (400 <code>CAMBIO_INVALIDO</code>); anti-IDOR : la marca/jornada referida debe pertenecer al empleado del token (403 <code>PROHIBIDO</code>)	solicitud creada, <code>estado:'pendiente'</code>	ninguno
POST /kiosk/terminos	<code>{token}</code>	token válido	<code>{aceptado, version, ts}</code>	ninguno
POST /kiosk/terminos/aceptar	<code>{token}</code>	token válido; idempotente	<code>{aceptado:true, version, ts}</code>	ninguno

Nota importante: en el canal kiosko la identidad es siempre la del **PIN introducido**, no el rol de sesión del host. Un empleado con rol `local_admin` o `technician` puede fichar en el kiosko como cualquier otro empleado; el módulo no distingue el rol en este canal (ver matriz §8).

4.2 Canal *manager* (requiere `session.resolve(req)` del host con rol `local_admin` / `technician`)

Método · Ruta	Body/Query	Validaciones	Respuesta / código	Rol requerido
GET /manager/ho y	—	sesión de host (401 <code>NO_AUTENTICADO</code>); rol (403 <code>PROHIBIDO</code>)	KPIs + marcas del día	<code>local_admin</code> , <code>technician</code>
GET /manager/reg istros	? desde&hasta&emp leadoId	rango por defecto = mes en curso	[{id,codigo,empl eadoId,...,marcas []}]	<code>local_admin</code> , <code>technician</code>
POST /manager/reg istros/marca /editar	{marcaId, tsNuevo, motivo}	<code>motivo</code> no vacío (400 <code>MOTIVO_REQUERIDO</code>); <code>tsNuevo</code> numérico (400 <code>TS_INVALIDO</code>); marca existente (404 <code>MARCA_INEXISTENTE</code>)	jornada recompuesta	<code>local_admin</code> , <code>technician</code>
POST /manager/reg istros/marca /anadir	{jornadaId, tipo, ts, motivo}	<code>tipo</code> ∈ {entrada,salida} (400 <code>TIPO_INVALIDO</code>); <code>ts</code> numérico (400 <code>TS_INVALIDO</code>); <code>motivo</code> no vacío (400 <code>MOTIVO_REQUERIDO</code>); jornada existente (404 <code>JORNADA_INEXISTENTE</code>)	jornada recompuesta	<code>local_admin</code> , <code>technician</code>
GET /manager/inf orme	? desde&hasta&emp leadoId	rango por defecto = mes en curso	informe agregado por empleado + total del periodo	<code>local_admin</code> , <code>technician</code>
GET /manager/inf orme.csv / .pdf	ídem	—	descarga	<code>local_admin</code> , <code>technician</code>
GET /manager/sol icitudes	? estado=pending e aprobada rech azada	—	lista	<code>local_admin</code> , <code>technician</code>
POST /manager/sol icitudes/:id /aprobar	{comentario?}	solicitud existente (404 <code>SOLICITUD_INEXISTENTE</code>); ya resuelta (409 <code>SOLICITUD_RESUELTA</code>)	aplica el cambio propuesto + versiona	<code>local_admin</code> , <code>technician</code>

Método · Ruta	Body/Query	Validaciones	Respuesta / código	Rol requerido
POST /manager/sol icitudes/:id /rechazar	{comentario?}	ídem	no cambia ningún dato	local_admin, technician
GET /manager/ajustes	—	—	config normalizada completa	local_admin, technician
PUT /manager/ajustes	{...claves}	claves desconocidas se ignoran; se normalizan tipos/rangos	config resultante	local_admin, technician
GET /manager/auditoria/verificar	—	—	{ok, rotoEn, total}	sólo technician
GET /manager/terminos	—	—	{aceptado, version, ts}	local_admin, technician
POST /manager/terminos/aceptar	—	idempotente	{aceptado:true,.. ..}	local_admin, technician

4.3 Códigos de error transversales

Code	HTTP	Origen
NO_AUTENTICADO	401	requireRol / requireAutenticado sin actor
PROHIBIDO	403	rol insuficiente, canal inválido, o IDOR (marca/jornada ajena)
CANAL_INVALIDO	403	ruta de kiosko invocada fuera del canal kiosk
SESION_KIOSKO	401	token de micro-sesión ausente/caducado (TTL 90 s)
RATE	429	límite de 30 peticiones/60 s superado (sólo en entrar y fichar)
PIN_INCORRECTO / PIN_BLOQUEADO	401 / 429	fallo de PIN / bloqueo por backoff
ERROR	500	cualquier excepción no ErrorPresentia (mensaje interno registrado en log, nunca en la respuesta)

El adaptador Fastify captura toda excepción; si es ErrorPresentia usa su status / code / publico , si no, responde 500 { code:'ERROR', mensaje:'Error interno.'

} y sólo registra en el log el `code` y el mensaje interno (nunca datos personales).

5. Lógica de negocio

5.1 Apertura y cierre de jornada; toggle entrada/salida

`fichar(deps, {empleadoId, pin, dispositivo, origen, pinVerificado})`
(`src/services/fichaje.service.js`):

1. Verifica que el empleado existe y está activo (403 `EMPLEADO_INVALIDO`).
2. Verifica el PIN si no viene ya verificado por la sesión de kiosko (`verificarPin` , ver §5.6).
3. Calcula la **fecha de jornada** en la zona horaria del centro (`fechaJornada(now, config.zonaHoraria)`), no en UTC.
4. Resuelve la jornada objetivo (`jornadaObjetivo` , ver §5.2): la del día si existe, o una jornada abierta reciente

(turno de noche), o ninguna (se creará una nueva).

1. Si no existe jornada, la crea con un código correlativo nuevo (`correlatives.next(serieCorrelativo, año)`).
2. Calcula el **tipo de marca siguiente** (`siguienteTipo`): `'entrada'` si no hay una entrada sin salida abierta,

`'salida'` en caso contrario.

1. Si `variasMarcasDia=false` y ya hay una entrada registrada hoy y toca fichar otra entrada ⇒ 409 `JORNADA_CERRADA` .
2. Inserta la marca, recalcula el estado de la jornada (`'cerrada'` si la última marca es una salida), audita el

evento `fichaje` , e imprime ticket si `imprimirTicket` está activo y el host aporta el puerto `printing` (best-effort: los fallos de impresión se ignoran).

Todo el bloque 4–8 corre dentro de una **transacción** (`SAVEPOINT` anidable, `src/db/tx.js`), con `ROLLBACK` si algo lanza.

5.2 Multi-marca (pausas) y turnos de noche

`jornadaObjetivo(deps, empleadoId, fecha, now)`:

- Si ya existe una jornada para (`empleadoId, fecha`), se reutiliza (permite varias parejas entrada/salida el mismo

día = pausas, si `variasMarcasDia` está activo).

- Si **no** existe jornada del día local pero el empleado tiene una **jornada abierta reciente** (su última marca es

una `entrada` de hace **menos de 24 h**, el fichaje (la salida) **cierra esa jornada anterior** en vez de crear una nueva del día siguiente. Esto es lo que permite que un turno de 23:00→02:00 compute correctamente sus 3 horas en la jornada de **entrada**, no en dos jornadas separadas de 0 horas cada una.

- Si la jornada abierta tiene **24 h o más** de antigüedad (olvido de días atrás), **no** se reutiliza: se abre una

jornada nueva hoy y la antigua queda huérfana, a la espera de corrección manual del admin (Registros o Solicitudes). No hay ninguna alerta proactiva de este caso (ver §14).

Este comportamiento corrigió un defecto real detectado en la auditoría previa del propio equipo (`DEF-001`, ver `auditoria/02-DEFECTOS.md`), verificado hoy con el test "*REGRESIÓN turno de noche*" (`test/auditoria.test.js`) y con `test/domain.test.js` ("jornada que cruza medianoche se computa correctamente": 22:00→00:00 = 120 min).

5.3 Cálculo de horas y redondeo

`emparejarSegmentos(marcas)` (`src/domain/jornadas.js`) ordena las marcas por `ts` y las empareja en segmentos `entrada→salida`; una segunda `entrada` sin `salida` previa cierra el segmento abierto anterior como "sin salida" (hueco); una `salida` sin `entrada` previa es un segmento huérfano que no computa. `minutosTrabajados()` suma sólo los segmentos cerrados con `salida > entrada`. El total se redondea con `aplicarRedondeo(min, redondeoMin)` (múltiplo más cercano; `redondeoMin=0` = sin redondeo) para producir `minutosRedondeados`, el valor que se muestra en Informe, Registros y exportaciones. **El ajuste `jornadaEstandarMin` no interviene en ningún cálculo**: se persiste y se muestra convertido a horas en Ajustes, pero no genera "horas extra" ni compara esperado/trabajado (confirmado leyendo `informe.service.js`; documentado por el propio equipo como `OBS-1`).

5.4 Correlativo `F-AAAA-NNNN`

`formatearCodigo(serie, anio, n)` → `F-2026-0001` (rellena a 4 dígitos; crece libremente a partir de 5). El número se obtiene de `correlatives.next(serie, año)`, atómico: si Expira no aporta este puerto, el fallback (`crearCorrelativosDb`) hace un único `UPSERT ... ON CONFLICT DO UPDATE SET ultimo = ultimo + 1 RETURNING ultimo` por serie+año — no hay ventana de carrera entre leer y escribir porque es una sola sentencia SQL.

5.5 Ciclo de vida de una solicitud de corrección

`presentia_solicitudes.estado`: `pendiente → aprobada` | `pendiente → rechazada` (sin otros estados ni transiciones; `aprobar` / `rechazar` sobre una solicitud ya resuelta devuelven 409 `SOLICITUD_RESUELTA`).

- **Crear** (`kiosk.crearSolicitud`): el empleado propone `{accion: 'editar' | 'anadir', ...}`; anti-IDOR verifica que

la marca/jornada referida sea suya.

- **Aprobar** (`manager.aprobar` , sólo admin/técnico): aplica el cambio propuesto (`aplicarCambio`) — si es

'editar', versiona la marca (conserva el valor original en `presentia_marca_versiones`); si es 'anadir', inserta una marca nueva marcada `editado=1` . Todo dentro de una transacción; audita `solicitud_aprobada` .

- **Rechazar: no toca ningún registro de jornada**, sólo marca la solicitud como `rechazada` ; audita

`solicitud_rechazada` .

5.6 Versionado de correcciones (`presentia_marca_versiones`)

`versionarMarcaTs(db, {marcaId, tsNuevo, motivo, autorId, ts})` : inserta primero la fila de historial con el `valor_anterior` (el `ts` que tenía la marca antes de tocarla) y **sólo después** hace el `UPDATE` sobre `presentia_marcas.ts` (y marca `editado=1`). El `motivo` es obligatorio en toda edición directa (`registros.service`) o corrección aprobada (`solicitudes.service`). No existe ninguna ruta que permita sobrescribir sin dejar rastro.

5.7 Auditoría encadenada

Cada llamada relevante (`fichaje` , `marca_editada` , `marca_anadida` , `solicitud_creada/aprobada/rechazada` , `ajustes_cambiados` , `pin_fallido` , `pin_bloqueado` , `terminos_aceptados`) inserta una fila en `presentia_auditoria` con `hash = sha256(prev_hash + serialización_canónica(payload))` . La serialización canónica ordena las claves del objeto recursivamente para que el hash sea determinista. `verificarIntegridad(db)` recorre la tabla completa y recalcula la cadena; sólo el `technician` puede invocar el endpoint que la expone (`GET /manager/auditoria/verificar`).

5.8 Verificación de PIN, bloqueo y backoff

`verificarPin` (si `config.exigirPin !== false`): comprueba primero si el par (`empleado` , `dispositivo`) está bloqueado (`presentia_pin_intentos.bloqueado_hasta > now` , 429 `PIN_BLOQUEADO` , auditado como `pin_bloqueado`); si no, llama a `pin.verify` . Si falla, incrementa `fallos` y calcula `bloqueado_hasta = now + backoffMs(fallos)` (`src/security/pin-policy.js` : umbral de 3 fallos, backoff `30s→1m→2m→5m→15m`), audita `pin_fallido` (nunca el PIN). Si acierta, borra la fila de intentos. `pin-policy.js` también clasifica PIN triviales (repetidos, secuencias, lista de comunes) para uso opcional en el alta de PIN por parte de Expira (el módulo no impone esta validación en tiempo de fichaje, sólo la ofrece como utilidad).

6. Manager, pantalla por pantalla

Componente raíz: `manager/PresentiaSection.jsx`. Se monta con `<PresentiaSection rol="local_admin" apiBase="/presentia" />`. La sesión es la del host (`fetch` con `credentials:'include'` ; el Manager **no** gestiona tokens).

Antes de mostrar cualquier pestaña, comprueba `GET /manager/terminos` ; si `aceptado=false` , muestra `AceptacionTerminos` (selector de documentos legales + checkbox obligatorio + botón "Aceptar y continuar" deshabilitado hasta marcar la casilla) y no deja pasar hasta `POST /manager/terminos/aceptar` .

Cabecera (`px-header`): título "Fichajes", subtítulo "Registro de jornada · control horario del equipo", `BotonTema` (icono + texto, cicla claro→oscuro→auto) e `InsigniaModo` ( "Modo admin" o "Modo técnico", **siempre** en azul de admin — nunca violeta, aunque sea técnico).

Navegación: 6 pestañas (`role="tablist"`), la de "Solicitudes" lleva un contador numérico (`px-tab__contador`) con el número de solicitudes pendientes, refrescado tras cada acción y al montar.

6.1 Hoy (`tabs/Hoy.jsx`)

- 3 KPIs: **Dentro ahora** (acentuado), **Salidas** (personas que ya ficharon hoy y ya se fueron — jornada cerrada),

Personas hoy (empleados distintos con al menos una marca).

- Tabla "Marcas del día" (Empleado, Tipo —insignia verde `Entrada` /roja `Salida` —, Hora en `--font-mono` , Código),

ordenada de más reciente a más antigua.

- **Autorefresco cada 15 s** (`setInterval` , se limpia al desmontar); no hay filtros ni acciones en esta pestaña.

6.2 Registros (`tabs/Registros.jsx`)

- Filtros: **Desde/Hasta** (por defecto, mes en curso), **Empleado** (`<select>` construido a partir de los

nombres presentes en la página cargada — no hay endpoint de plantilla de empleados).

- Tabla: Empleado, Fecha, Entrada, Salida (insignia ámbar "En curso" si falta), Código, insignia "Editado" si

`editado=true` , columna de acciones.

- Acción **Editar** (modal): selector de marca de la jornada (con "(editada)" si corresponde), `<input datetime-local>` con la nueva hora, `textarea` de motivo obligatorio; llama a `POST /manager/registros/marca/editar``.

- Acción **Añadir marca** (modal): tipo (entrada/salida), hora, motivo obligatorio; llama a ``POST /manager/registros/marca/anadir``.

- **Exportar CSV:** generado **en el cliente** a partir de las filas ya cargadas (no hay endpoint dedicado de

exportación de Registros, a diferencia de Informe).

- Toast de confirmación ("Marca actualizada."/"Marca añadida.") tras cada acción, y recarga automática de la tabla.

6.3 Informe de horas (`tabs/InformeHoras.jsx`)

- Mismos filtros de rango + empleado que Registros.
- Un bloque por empleado con su nombre, insignia "Total: `168 h 30 m`" y tabla de jornadas (Fecha, Código, Entrada,

Salida —o "En curso"—, Horas).

- Caja "Total del periodo" al final, agregando todos los empleados visibles.
- **Exportar CSV** y **Exportar PDF:** ambos son **descargas del servidor** (`GET /manager/informe.csv|.pdf`), no

generados en cliente.

6.4 Solicitudes (`tabs/Solicitudes.jsx`)

- Sub-pestañas internas: **Pendientes / Aprobadas / Rechazadas** (con contador propio en "Pendientes").
- Cada línea: nombre del empleado, insignia neutral con el `tipo` (`correccion`), insignia de estado (ámbar

"Pendiente" / verde "Aprobada" / roja "Rechazada"), descripción textual del cambio propuesto (`describirCambio` : "Añadir marca (entrada) a las 18:30" / "Editar marca (salida) a las 17:45"), y el motivo.

- Sólo las pendientes muestran botones **Aprobar/Rechazar**, que abren un modal con comentario opcional antes

de confirmar (`POST /manager/solicitudes/:id/aprobar|rechazar`).

- Tras resolver, se refresca la lista y el contador de pendientes global (que también actualiza el badge de la

pestaña en la cabecera del Manager).

6.5 Ajustes (`tabs/Ajustes.jsx`)

Sólo expone **7 de las 10 claves** de configuración (ver detalle exacto en §9): `jornadaEstandarMin` (número, con conversión a horas mostrada como ayuda), `redondeoMin` (número, "0 = sin redondeo"), `temaPorDefecto` (select claro/oscuro/automático) y 4 interruptores (`Toggle` , `role="switch"`): "Mostrar «Fichar» en el kiosko", "Exigir PIN", "Varias marcas por día", "Imprimir ticket", cada uno con su descripción de una línea. Botones **Descartar** (recarga desde el servidor) y

Guardar cambios (`PUT /manager/ajustes` , sólo con las 7 claves editables; el servidor normaliza y devuelve la config resultante, que sustituye al formulario para reflejar cualquier ajuste de rango). `zonaHoraria` , `conservacionAnios` y `serieCorrelativo` **no tienen control en esta pantalla** — sólo son modificables llamando directamente a la API.

6.6 Legal (`tabs/Legal.jsx`)

Visor de los 10 documentos legales (fuente `legal/*.md` , embebidos en `legal/contenido.js` por `legal/generar-contenido.mjs`), agrupados por categoría (Protección de datos · Legales · Cumplimiento) en un aside con buscador de texto libre. Cada documento se renderiza con el parser Markdown propio (`shared/Markdown.jsx` , sin `dangerouslySetInnerHTML`). Toolbar con **Imprimir** (`window.print()`) y **Descargar .md** (`Blob` + enlace temporal). Sólo lectura: los textos se editan en los `.md` fuente y se regeneran con `npm run legal:build` .

7. Kiosko

Componentes: `kiosk/FicharCard.jsx` (tarjeta verde del menú del kiosko, con prop `visible` que el host debe atar al ajuste `mostrarEnKiosko` — el propio componente no consulta la API de ajustes) y `kiosk/FicharScreen.jsx` (pantalla completa de fichaje), montados por el host del kiosko (no hay enrutador propio).

Identidad y sesión: `crearApiKiosk({base, dispositivo})` envía siempre la cabecera `x-presentia-dispositivo` . El **token de micro-sesión** vive sólo en estado de React (memoria), nunca en `localStorage` ni cookies; caduca a los ~90 s. Si cualquier llamada devuelve `401 SESSION_KIOSKO` , la pantalla vuelve automáticamente al paso de PIN conservando el empleado seleccionado.

Flujo de pantallas (`vista` en `FicharScreen`):

1. `empleados` — grid de tarjetas (avatar o iniciales) desde `GET /kiosk/empleados` ; enlace a "Información legal

y protección de datos".

1. `pin` — cabecera "Hola, {nombre}", puntos de PIN (mínimo 4, máximo 8 dígitos), teclado numérico 0-9 +

Borrar + Entrar (deshabilitado si `pin.length < 4` o hay envío en curso). `POST /kiosk/entrar` .

1. `aceptar` (sólo la primera vez del empleado, si `aceptado=false` en la respuesta de `entrar`)
—

`AceptacionTerminos` compartido con el Manager.

1. `panel` — avatar, "Hola, {nombre}", insignia Dentro/Fuera (con hora de entrada si está dentro), reloj en

vivo (`HH:MM:SS` , actualizado cada segundo) y fecha larga en español, **botón único enorme** que alterna **FICHAR ENTRADA** (verde) / **FICHAR SALIDA** (rojo) según `siguienteTipo` ; toast con la hora tras fichar. Accesos a "Ver mis registros", "Información legal" y "Terminar".

1. `registros` (`MisRegistros.jsx`) — selector de rango (mes en curso por defecto), tabla de jornadas propias,

total del periodo, **Exportar CSV/PDF** (descargas con el token en la query string).

1. `legal` (`InfoLegal.jsx`) — sólo los documentos marcados `empleado:true` en `legal/contenido.js` (2 de los 10:

cláusula informativa y política de privacidad), accesible con o sin sesión activa.

Aviso de tratamiento (`AvisoTratamiento.jsx`): overlay RGPD mostrado la primera vez que se abre el kiosko en un dispositivo (aceptación persistida en `localStorage` , **nunca** el token de sesión); resume responsable, finalidad, datos tratados, ausencia de biometría/geolocalización y conservación de 4 años, con enlace para ver la cláusula completa.

Antirrebote: el botón de fichar se deshabilita mientras hay una petición en curso (mitigación en **cliente**; ver limitación de servidor en §14).

8. Roles y permisos

Roles definidos (`ROLES` en `ports.js`): `empleado` , `local_admin` , `technician` . No existe "supervisor" ni "superadmin". El canal **kiosk** no consulta el rol de la sesión del host en absoluto: la identidad es la del PIN introducido y las acciones se limitan a la propia identidad (anti-IDOR reforzado en servidor). El canal **manager** exige `actor.rol ∈ {local_admin, technician}` en cada endpoint (comprobado en servidor, nunca confiado del cliente).

Acción / endpoint	<code>no_auth</code> (sin sesión host)	<code>empleado</code>	<code>local_admin</code>	<code>technician</code>
<code>GET /kiosk/empleados</code>	✓ (canal <code>kiosk</code> , sin rol)	✓	✓	✓
<code>POST /kiosk/entrar</code> (con su propio PIN)	✓	✓	✓	✓
<code>POST /kiosk/fichar</code> , <code>estado</code> , <code>mis-registros</code> , <code>terminos*</code>	✓ (con token válido)	✓	✓	✓
<code>POST /kiosk/solicitud</code> (sobre su propia jornada)	✓	✓	✓	✓
<code>POST /kiosk/solicitud</code> sobre jornada/marca ajena	✗ 403 <code>PROHIBIDO</code> (anti-IDOR, para cualquier identidad)	✗	✗	✗
<code>GET/POST /manager/*</code> (cualquier ruta de Manager)	✗ 401 <code>NO_AUTENTICADO</code>	✗ 403 <code>PROHIBIDO</code>	✓	✓

Acción / endpoint	no_auth (sin sesión host)	empleado	local_admin	technician
GET <code>/manager/auditoria/verificar</code>	✗ 401	✗ 403	✗ 403 PROHIBID 0	✓ (único rol permitido)
Ver/editar sus propios registros (kiosko)	—	✓ (siempre lo propio)	✓	✓
Ver/editar registros de cualquier empleado (Manager)	✗	✗	✓	✓
Cambiar <code>Ajustes</code> del módulo	✗	✗	✓	✓

Notas de la matriz:

- Un empleado con rol `local_admin` / `technician` en Expira que ficha en el kiosko **con su propio PIN** se comporta

como cualquier `empleado` desde el punto de vista del canal kiosko (el módulo no distingue rol allí).

- "Ocultar un botón no autoriza": la UI del Manager no vuelve a comprobar el rol antes de mostrar acciones; toda la

autorización real vive en `authz.js` (`requireRol`) y se aplica en **cada** handler, no una vez por sesión.

- El canal (`kiosk` vs `manager`) no lo decide el cliente: está fijado por la ruta registrada en

`fastify-adapter.js` (`run(handler, 'kiosk' | 'manager', ...)`); no existe un parámetro de "canal" que el cliente pueda manipular para escalar de kiosko a Manager.

9. Configuración

Persistida en `presentia_ajustes` (clave/valor JSON); normalizada siempre por `normalizeConfig()` (tipos, mínimos, máximos). El objeto vivo `deps.config` se reemplaza in-place tras cada `PUT /manager/ajustes` (`Object.assign(deps.config, nueva)`), de forma que servicios que ya capturaron `deps.config` ven el cambio sin reiniciar.

Clave	Tipo / valores admitidos	Por defecto	Efecto real	Editado en Ajustes (UI)
<code>zonaHoraria</code>	string IANA (p. ej. <code>Europe/Madrid</code>)	<code>Europe/Madrid</code>	Determina la fecha de jornada (<code>fechaJornada</code>), el año del correlativo, y todo formateo de horas (kiosko, CSV, PDF). Ningún control de tipo en <code>normalizeConfig</code> (se acepta cualquier string; un valor inválido rompería <code>Intl.DateTimeFormat</code> en tiempo de uso)	No (sólo vía API)

Clave	Tipo / valores admitidos	Por defecto	Efecto real	Editabile en Ajustes (UI)
<code>jornadaEstandarMin</code>	entero 0–1440 (min)	480 (8 h)	Ninguno en el cálculo: sólo se muestra convertido a horas en Ajustes. No genera horas extra ni compara esperado/trabajado	Sí
<code>redondeoMin</code>	entero 0–120 (min)	0	Redondea <code>minutosRedondeados</code> al múltiplo más cercano; afecta a Informe, Registros y exportaciones	Sí
<code>mostrarEnKiosko</code>	boolean	<code>true</code>	<code>false</code> ⇒ <code>GET /kiosk/empleados</code> devuelve <code>[]</code> (nadie puede fichar); la tarjeta <code>FicharCard</code> sólo se oculta si el host ata su propio <code>visible</code> a este ajuste — no ocurre automáticamente	Sí
<code>exigirPin</code>	boolean	<code>true</code>	<code>false</code> ⇒ <code>verificarPin</code> no llama a <code>pin.verify</code> en absoluto y siempre devuelve <code>true</code> : <code>POST /kiosk/entrar</code> acepta cualquier valor de <code>pin</code> (incluido vacío) para cualquier <code>empleadoId</code> activo	Sí
<code>variasMarcasDial</code>	boolean	<code>true</code>	<code>false</code> ⇒ tras la primera pareja entrada/salida del día, un nuevo intento de <code>entrada</code> devuelve 409 <code>JORNADA_CERRADA</code>	Sí
<code>imprimirTicket</code>	boolean	<code>false</code>	<code>true</code> ⇒ tras fichar, invoca <code>deps.printing.printTicket(...)</code> si el host aportó ese puerto opcional; si no lo aportó, no ocurre nada (silencioso, sin error)	Sí
<code>conservacionAnios</code>	entero, mínimo forzado 4	4	Ninguno en tiempo de ejecución: no hay ningún job/endpoint de purga que lo lea; es puramente documental (referenciado en <code>legal/POLITICA-DE-CONSERVACION-Y-SUPRESION.md</code>)	No
<code>serieCorrelativo</code>	string	<code>F</code>	Prefijo usado por <code>correlatives.next(serie, año)</code> al generar el código de cada jornada nueva	No
<code>temaPorDefecto</code>	<code>'claro'</code> <code>'oscuro'</code> <code>'auto'</code>	<code>auto</code>	Se pasa como prop <code>defecto</code> a <code>BotonTema</code> en el Manager; sólo se adopta si el dispositivo no tiene ya una preferencia guardada en <code>localStorage</code> . El kiosko no lee ni usa esta clave en ningún componente (usa siempre <code>auto</code> de forma fija, D-015)	Sí (afecta sólo al Manager)

10. Diseño y temas

10.1 Tokens (`shared/tokens.css` , única fuente de verdad)

- **Paleta primitiva:** base *slate* (superficie/borde/texto), **azul** = admin y técnico (`--blue-action #2f5fbf`), el técnico usa el mismo azul, nunca violeta), **verde** = empleado (`--green-action #1e9a59`), **rojo** = peligro (`--red-action #dc2626`). Estados con significado constante: verde=correcto, ámbar=pendiente/en curso, rojo=error.

- **Alias semánticos:** `--color-`, `--status-`, `--dot-*` referencian siempre las primitivas; los componentes

sólo usan alias, nunca hex directos (verificado por `test/tema.test.js`, "CERO colores hardcoded fuera de `tokens.css`").

- **Radios:** `--radius-xs 6px` ... `--radius-full 9999px`. **Sombras:** 4 niveles (`xs/card/soft/modal`).

Movimiento: `--duration-fast/normal/slow` + easings estándar.

- **Tipografía:** `--font-sans = "Inter", "Segoe UI", Tahoma, system-ui, sans-serif`; `--font-mono` para datos

técnicos (horas, códigos). **Inter está declarada pero nunca se carga** en la práctica — ver §14.

10.2 Tema oscuro

Bloque único `:root[data-tema-efectivo="oscuro"]` que **redefine sólo las primitivas**; los alias siguen por cascada (ningún componente sabe qué tema hay activo). Contraste verificado por `test/tema.test.js` y documentado en `docs/TEMA-OSCURO-ANALISIS.md`: mínimo **4.89:1** en texto atenuado sobre superficie elevada y **3.96:1** en elementos de interfaz con foco — ambos por encima de AA (4.5:1 texto normal / 3:1 UI). En oscuro, el texto sobre botones llenos pasa a oscuro (`--color-text-on-accent:#0f172a`) porque las acciones son más luminosas; los enlaces usan un token propio (`--color-enlace:#7ea6f0`) para mantener AA sin depender del alias de acción azul.

10.3 Modelo de tema: dos atributos en `<html>`

`data-tema` (preferencia: `claro|oscuro|auto`) y `data-tema-efectivo` (resuelto: `claro|oscuro`, lo único que lee el CSS). `shared/tema.js` resuelve `auto` vía `matchMedia('(prefers-color-scheme: dark)')` y escucha cambios en caliente. Persistencia por dispositivo en `localStorage` (`presentia.tema`); si el dispositivo no tiene preferencia propia, adopta `temaPorDefecto` de Ajustes. Requiere un script anti-FOUC en el `<head>` del host (documentado en `docs/TEMA-OSCURO.md`) para fijar el tema antes del primer render. El kiosko no lleva botón de tema (dispositivo compartido) y usa siempre `auto`. El PDF exportado se genera siempre en claro.

10.4 Responsive (`shared/responsive.css`, **mobile-first**)

- **Breakpoints:** 480 / 640 / 1024 / 1440 px (D-016); el corte tabla↔tarjetas es 640 px.
- **Objetivos táctiles** ≥ 44 px e **inputs** ≥ 16 px (evita el zoom de iOS), pero **sólo bajo** ``@media`

`(any-pointer:coarse)`` — con ratón fino (escritorio/Electron) se conserva la densidad de tabla original; es decir, el mismo componente cambia de tamaño según el tipo de puntero, no según el ancho de pantalla.

- **Tablas de datos** → **tarjetas apiladas** en ≤ 640 px (`display:block` + `td::before{content:attr(data-label)}`);

ningún dato ni acción desaparece. Las tablas de **prosa** (Legal, contenido Markdown arbitrario) mantienen scroll horizontal contenido en vez de tarjetizarse (D-013).

- **Modales** → **hoja inferior** en ≤ 640 px, con pie de acciones `sticky` y `env(safe-area-inset-bottom)` para

notch/gestos (D-018). Alturas en `dvh` con respaldo `vh`.

- **Filtros** del Manager se apilan a ancho completo en columna (nunca amontonados) en ≤ 640 px (D-017).

10.5 Componentes visuales reutilizables

`Insignia.jsx` (7 variantes con significado de color constante: `exito`, `aviso`, `en-curso`, `info`, `peligro`, `editado`, `neutral`; sin `dangerouslySetInnerHTML`), `Toast.jsx` (auto-descarte a 3.5 s), `Markdown.jsx` (renderiza Markdown a nodos React reales — negrita, cursiva, código, enlaces, listas, tablas, citas, bloques de código — nunca HTML crudo, ver `shared/markdown-parse.js`).

11. Flujos de uso

11.1 Jornada normal (entrada → dentro en Hoy → salida → Registros con horas)

Empleado entra al kiosko, elige su tarjeta, introduce PIN (`POST /kiosk/entrar`) → recibe token + `estado.dentro = false`. Pula **FICHAR ENTRADA** (`POST /kiosk/fichar`) → se crea la jornada con código `F-2026-0001`, marca `entrada`, jornada `abierta`. En el Manager, "Hoy" muestra `dentroAhora=1`. Horas después, el mismo empleado vuelve a entrar con su PIN y pulsa **FICHAR SALIDA** → misma jornada (mismo código), marca `salida`, jornada `cerrada`. "Hoy" pasa a `dentroAhora=0`, `salidas=1`. En "Registros", la fila muestra `enCurso:false`, minutos correctos y `editado:false`. (Verificado end-to-end en `test/flujos.test.js`, flujo (a)).

11.2 Olvido de fichaje → solicitud → aprobación → corrección versionada

El empleado sólo ficha la entrada y se marcha sin fichar la salida. Desde el kiosko, crea una solicitud (`POST /kiosk/solicitud`, `cambio:{accion:'anadir', jornadaId, tipo:'salida', ts}`, motivo obligatorio) → queda `pendiente`. El admin la ve en Solicitudes → **Aprobar** con comentario opcional (`POST /manager/solicitudes/:id/aprobar`) → se inserta la marca de salida marcada `editado=1`, la jornada pasa a `cerrada` con las horas correctas, y la solicitud queda `aprobada`. Si en vez de aprobar el admin **rechaza**, no se modifica ningún registro de jornada (verificado: el nº de marcas antes/después es idéntico).

11.3 Corrección directa del admin (sin pasar por solicitud)

Desde Registros, el admin abre "Editar marca" sobre una jornada existente, cambia la hora y escribe el motivo → `POST /manager/registros/marca/editar`. El valor **original** queda conservado en

`presentia_marca_versiones` (una fila con `valor_anterior` = el ts previo exacto) antes de que la marca cambie; la jornada queda marcada `editado`.

11.4 Turno de noche (23:00 → 02:00)

Entrada a las 23:00 (jornada del día D, código `F-AAAA-NNNN`). El empleado ficha salida a las 02:00 del día D+1: como no existe jornada de D+1 pero sí una jornada abierta de D con menos de 24 h, la salida **cierra la jornada de D** (no crea una de D+1). El informe de D muestra 3 h correctas; D+1 no tiene ninguna jornada espuria.

11.5 Primer acceso (aceptación de términos)

Tanto un empleado nuevo en el kiosko como un admin/técnico nuevo en el Manager ven, antes de cualquier otra pantalla, el gate de aceptación de términos (todos los documentos legales, checkbox obligatorio). Tras aceptar (`.../terminos/aceptar`), queda registrado en `presentia_aceptaciones` y auditado; no se vuelve a pedir mientras `TERMINOS_VERSION` no cambie.

11.6 Verificación de integridad de la auditoría (sólo técnico)

El técnico entra en el Manager, y sólo él puede llamar `GET /manager/auditoria/verificar`; el resultado `{ok:true/false, rotoEn}` indica si alguna fila fue alterada desde el génesis de la cadena. (Ver limitación real en §14: no cubre el borrado de las últimas filas.)

12. Integración en Expira

Procedimiento completo en `INTEGRACION-EN-EXPIRA.md` (8 pasos). Resumen:

1. **Copiar** la carpeta del módulo dentro del repo de Expira; verificar `npm test` en verde.
2. **Registrar el backend:** `crearModulo(deps)` + `registrarFastify(fastify, modulo, {prefix:'/presentia'})`, con

los puertos reales de Expira (ver tabla §2.2).

1. **Puertos obligatorios:** `db`, `clock.now`, `employees`, `session.resolve`, `pin.verify` (si `exigirPin`);

opcionales con fallback propio: `correlatives`, `printing`, `hash`.

1. **Migración** (tras copia de seguridad de la BD de Expira): aditiva e idempotente, 9 tablas `presentia_*`, nunca

toca tablas preexistentes.

1. **Manager:** añadir la entrada "Presentia" al sidebar y montar `<PresentiaSection rol={rol} apiBase="/presentia" />`.
2. **Kiosko:** añadir la tarjeta "Fichar" y montar `<FicharScreen api={apiKiosk} />`.
3. **Tema:** incluir el script anti-FOUC en el `<head>` del host.

4. **Smoke test:** `npm run smoke` debe imprimir `SMOKE TEST: OK` (ejercita migración, PIN, entrada/salida,

Registros, horas, informe, kiosko y auditoría contra los puertos reales).

Plan de reversión: quitar la llamada a `registrarFastify` y el montaje de los componentes React deja a Expira funcionando exactamente igual que antes; las tablas `presentia_*` son aditivas y pueden dejarse o eliminarse manualmente (nunca automáticamente).

Pendientes que sólo pueden cerrarse contra el Expira real (`TODO-INTEGRACIÓN`, ver `INTEGRACION-EN-EXPIRA.md`): regresión funcional de Expira (etiquetado, caducidades, licencia, Hub, backups, impresión — no verificable sin su código), cifrado en reposo y de backups, servidor en `127.0.0.1` + TLS si hay acceso por red, CSP y cabeceras, endurecimiento de Electron (`contextIsolation`, sin `nodeIntegration`, `sandbox`), NTP en el dispositivo del kiosko, empaquetado de la fuente Inter, y validación visual en hardware real.

13. Glosario

- **Jornada:** el registro de un día de un empleado (`presentia_jornadas`); agrupa todas sus marcas de ese día en

la zona horaria del centro.

- **Marca:** un fichaje individual de tipo `entrada` o `salida` (`presentia_marcas`), con timestamp UTC.
- **Segmento:** un par entrada→salida emparejado por el dominio (`emparejarSegmentos`); una jornada puede tener

varios segmentos (pausas).

- **Código correlativo** (`F-AAAA-NNNN`): identificador único de una jornada, serie configurable + año + número

secuencial por año.

- **Solicitud:** propuesta de corrección hecha por un empleado, sujeta a aprobación/rechazo del admin.
- **Versión (de marca):** entrada de historial que conserva el valor anterior de una marca corregida; nunca se

sobrescribe ni se borra.

- **Auditoría encadenada por hash:** log append-only donde cada fila incluye el hash de la anterior, de modo que

alterar una fila intermedia rompe la cadena de forma detectable.

- **Canal:** `kiosk` (identidad por PIN + micro-sesión) o `manager` (identidad por sesión del host); fijado por la

ruta, no por el cliente.

- **Micro-sesión de kiosko:** token efímero (TTL ~90 s) emitido tras un PIN correcto; sólo se guarda su hash en el

servidor y el token en memoria de React en el cliente.

- **Puerto:** interfaz que el módulo espera que Expira implemente (p. ej. `db`, `employees`, `session`); patrón

ports & adapters.

- **ErrorPresentia**: excepción de dominio con `code` estable, `status` HTTP, mensaje interno (log) y mensaje

público genérico (nunca revela detalles sensibles).

- **Aceptación de términos:** registro de consentimiento informado (quién, cuándo, qué versión) exigido una vez por

usuario en el primer acceso.

14. Observaciones / deuda

Hallazgos verificados directamente sobre el código en esta revisión (no son una copia de la auditoría previa del propio equipo, aunque coinciden en dos puntos que se referencian explícitamente):

1. **Las claves foráneas no se hacen cumplir.** `presentia_marcas.jornada_id` es la única `FOREIGN KEY` declarada en

todo el esquema (`src/db/schema.js`); `presentia_marca_versiones.marca_id` y `presentia_solicitudes.jornada_id/marca_id` son referencias puramente lógicas, sin `FOREIGN KEY` ni `CHECK`. Y ni siquiera la única FK declarada se aplica de verdad: no hay ningún `PRAGMA foreign_keys = ON` en el código del módulo ni en `src/dev/reference-env.js` (verificado por búsqueda en todo `src/`). En SQLite las FK están **desactivadas por defecto** salvo que la conexión lo active explícitamente, así que salvo que Expira la active por su cuenta, la integridad referencial de estas tablas depende sólo de la disciplina del código de aplicación.

1. **La cadena de auditoría no detecta el truncamiento del final.** `verificarIntegridad()` recorre

`SELECT * FROM presentia_auditoria ORDER BY id ASC` y recalcula `hash` fila a fila; esto detecta perfectamente la alteración de una fila intermedia (probado: `test/seguridad.test.js`, "alterar una línea rompe la cadena"), pero si alguien con acceso directo a la BD **borra las últimas N filas** (`DELETE ... WHERE id > X`), la cadena restante sigue siendo internamente consistente y `verificarIntegridad()` devuelve `ok:true` sin detectar nada: no hay ningún ancla externa (recuento esperado de filas, timestamp firmado externamente, replicación) que permita detectar la

desaparición de eventos recientes. Ninguna de las 121 pruebas ejercita este escenario (sólo se prueba `UPDATE`, nunca `DELETE` sobre filas finales).

1. `jornadaEstandarMin` es puramente decorativo (confirmado leyendo `informe.service.js` y toda la cadena de

cálculo): se persiste, se valida y se muestra convertido a horas en Ajustes, pero no interviene en ningún cálculo de horas extra ni de "esperado vs. trabajado". Ya documentado por el propio equipo como `OBS-1` en `auditoria/02-DEFECTOS.md`; se confirma que sigue siendo así.

1. `conservacionAnios` tampoco tiene efecto real en tiempo de ejecución. Se fuerza a un mínimo de 4 (nunca

rebajable) en `normalizeConfig`, y así lo documenta la política legal (`legal/POLITICA-DE-CONSERVACION-Y-SUPRESION.md`), pero no existe ningún job, endpoint o consulta en todo `src/` que lo lea para purgar o bloquear datos vencidos — es un valor de configuración sin consumidor. Si nadie lo implementa nunca contra Expira, los datos de jornada crecerán indefinidamente más allá del plazo legal sin ningún mecanismo de purga.

1. **Rate limiting cubre sólo 2 de los ~24 endpoints.** `crearRateLimiter` (30 peticiones/60 s) se invoca

únicamente en `kiosk.entrar` y `kiosk.fichar` (confirmado en `handlers.js`); `kiosk.crearSolicitud`, `kiosk.misRegistros`, `kiosk.exportar` y **absolutamente ningún** endpoint de `manager.*` (incluidos `ajustesPut`, `aprobar`, `rechazar`, las exportaciones y la propia verificación de auditoría) tienen límite de ritmo. Documentado parcialmente por el propio equipo (`OBS-2`), pero el alcance real (cero límite en todo el canal Manager) es más amplio de lo que sugiere esa nota.

1. `fichar` es un toggle sin ninguna protección de servidor contra doble envío. Dos peticiones casi simultáneas

con el mismo token válido producen una entrada y una salida separadas por milisegundos (jornada de 0 minutos), sin ningún candado transaccional ni deduplicación por servidor; la única mitigación es deshabilitar el botón en el cliente (`FicharScreen`, variable `enviando`), que no protege contra una segunda pestaña, un doble toque real en pantallas táctiles con lag, o un reintento automático de red. El rate-limit de 30/60 s tampoco está pensado para detectar esto (documentado por el propio equipo como `OBS-4`, aceptado como riesgo de diseño).

1. **La tipografía normativa (Inter) nunca se carga.** `shared/tokens.css` declara `--font-sans` con `"Inter"` como

primera opción y `shared/fonts.css` prepara los `@font-face` correspondientes, pero (a) `fonts.css` **no se importa desde ningún componente ni hoja de estilos** del proyecto (búsqueda confirmada: cero referencias fuera de sí mismo), y (b) aunque se importara, apunta a `./fonts/Inter-Regular.woff2` (y Medium/SemiBold/Bold), y el directorio `shared/fonts/` **no existe** en el repositorio. En la práctica, todo el sistema (Manager y kiosko) renderiza hoy con el fallback `"Segoe UI"`, silenciosamente, pese a que la tipografía se documenta como normativa.

1. `editarMarca` (admin) no valida coherencia entre el nuevo `ts` y la `fecha` de la jornada que edita. Un admin

puede fijar cualquier fecha/hora arbitraria (incluso de otro año) en una marca de una jornada concreta sin que el servidor lo impida ni lo señale; queda auditado y con motivo, pero la fila `presentia_jornadas.fecha` no se recalcula ni se contrasta. Documentado como decisión consciente por el propio equipo (`OBS-3`), no como defecto, pero sigue siendo una laguna de integridad de datos que un futuro admin descuidado puede introducir sin aviso.

1. **Jornadas abiertas "huérfanas" (turno de noche de ≥ 24 h) no generan ninguna alerta.**

Cuando `jornadaObjetivo`

decide no reutilizar una jornada abierta de hace 24 h o más (para no cerrar por error un olvido antiguo), esa jornada queda indefinidamente `abierta` y en curso, sin salida y sin horas computadas, y **no aparece destacada** en ningún KPI de "Hoy" ni recibe ningún aviso proactivo en el Manager — sólo se detecta si un admin revisa manualmente "Registros" o si el propio empleado la reporta.

1. **Los 10 documentos legales se sirven con marcadores sin rellenar** (`[NOMBRE_EMPRESA]`, `[CIF]`, `[DOMICILIO]`,

`[EMAIL_DPD]`, `[NOMBRE_PROVEEDOR]`, `[HUB_CENTRAL]`) tal cual, tanto en el visor del Manager (`Legal.jsx`) como en el kiosko (`InfoLegal.jsx`, `AvisoTratamiento.jsx`); un empleado real vería hoy, literalmente, "Responsable: **[NOMBRE_EMPRESA]**" si el módulo se desplegara sin completar antes esos campos y sin revisión de un abogado/DPD (el propio `legal/CUMPLIMIENTO.md` ya lo marca como `TODO-LEGAL` pendiente, junto con el hecho de que el Real Decreto de desarrollo del art. 34.9 ET seguía sin publicarse en el BOE a la fecha de redacción, 2026-07-13).

Documento generado en la Fase 1 de la revisión integral de Presentia. Fuente editable:

`docs/DOCUMENTACION-PRESENTIA.md`. Evidencias verificadas: lectura completa de `src/`, `manager/`, `kiosk/`, `shared/`, `Legal/`, `test/`, `auditoria/` y `docs/` preexistente; ejecución de `npm test` (121/121 en verde); comprobación en vivo de `http://127.0.0.1:8787`.

Generado el 2026-07-19 · agente de documentación · fuente editable: docs/DOCUMENTACION-PRESENTIA.md